信道编码码：PHY

LM SEC：APP

] → CL-SEC.

CL-SEC: Cross-Layer Semantic Error Correction Empowered by Language Models

Yirun Wang, Yuyang Du, Soung Chang Liew, Yuchen Pan, Feifan Zhang, and Lihao Zhang
The Department of Information Engineering, The Chinese University of Hong Kong
Shatin, New Territories, Hong Kong Special Administrative Region

Abstract

Achieving reliable communication has long been a fundamental challenge in networked systems. Semantic Error Correction (SEC) leverages the semantic understanding capabilities of language models (LMs) to perform application-layer error correction, complementing conventional channel decoding. While promising, existing SEC approaches rely solely on context captured by LMs at the application layer, ignoring the rich information available at the physical layer. To address this limitation, this paper introduces Cross-Layer SEC (CL-SEC), an LM-empowered error correction framework that integrates cross-layer information from both the physical and application layers to jointly correct corrupted words in text communication. Using a Bayesian combination in product form tailored to this framework, CL-SEC achieves significantly improved performances over methods that process information in isolated layers. CL-SEC shows substantial gains across multiple error-correction metrics, including bit-error rate, word-error rate, and semantic fidelity scores. Importantly, unlike most semantic communication systems that focus solely on recovering the semantic meaning of transmitted messages, CL-SEC aims to reconstruct the original transmitted message verbatim, leveraging the semantic understanding capabilities of LMs for precise reconstruction.

CCS Concepts

• **Networks** → **Network reliability**; **Error detection and error correction**.

Keywords

Error correction, Language models, Semantic communication, Semantic error correction, Cross-layer error correction

ACM Reference Format:
Yirun Wang, Yuyang Du, Soung Chang Liew, Yuchen Pan, Feifan Zhang, and Lihao Zhang. 2026. CL-SEC: Cross-Layer Semantic Error Correction Empowered by Language Models. In *Proceedings of The 27th International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing (MobiHoc '26).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

MobiHoc '26, Tokyo, Japan

ACM ISBN 979-8-4007-XXXX-X/2026/11
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Error correction has been an enduring research topic in networked systems. While demanding high reliability, networked systems remain inherently susceptible to transmission errors. Classical channel coding approaches introduce controlled redundancy into transmitted data to enable forward error correction (FEC) at the receiver [8, 10, 12]. These techniques allow the receiver to detect and correct transmission errors at the physical layer by exploiting statistical relationships within the encoded message. While these physical-layer methods have been proven highly effective for error correction and serve as the cornerstone of modern communication systems, they may still encounter unrecoverable errors under highly adverse channel conditions. Consequently, the quest for more robust error correction methods has remained an active and ongoing focus of the research community.

The rapid development of language models (LMs) ushers in a new approach to tackling the error-correction problem. Leveraging semantic understanding acquired from massive training datasets, LMs can identify corrupted text data by recognizing linguistic patterns and revise it based on contextual coherence. Unlike conventional physical-layer methods, which rely on signal-level information, LMs exploit semantic and linguistic patterns to correct errors in a way that reflects the logic and structure of human communication. This provides a complementary approach to traditional FEC by addressing errors at the application layer through a deeper understanding of language.

To illustrate, consider Fig. 1, where a corrupted message is received (see the second box). After FEC at the physical layer, some errors may persist if the message has been severely corrupted (see the third box). Semantic Error Correction (SEC) leverages LMs to address these residual errors. For example, an LM can inspect the message, identify a corrupted word such as "*syategs*", and revise it to the correct word "*systems*" (see "Standalone SEC" in the fourth box). This approach – utilizing LMs to correct erroneous words – is a basic SEC method performed after FEC, and it provides an additional layer of error correction to handle remaining errors.

To motivate our approach, let us analyze the limitations of standalone SEC following FEC. Operating exclusively at the application layer, standalone SEC may still encounter problems in recovering the transmitted message under severely corrupted receptions. In particular, standalone SEC may suffer from two key shortcomings: ***semantic inaccuracy*** and ***lexical imprecision***.

- ***Semantic Inaccuracy:*** Standalone SEC may produce outputs that are semantically plausible but factually incorrect. For instance, in Example A (see "Standalone SEC" in the last box in Fig. 1), the recovered word "*encrypted*" is contextually coherent with the corrupted sentence in the third box but fails to capture the intended meaning of the original word "*corrupted*".

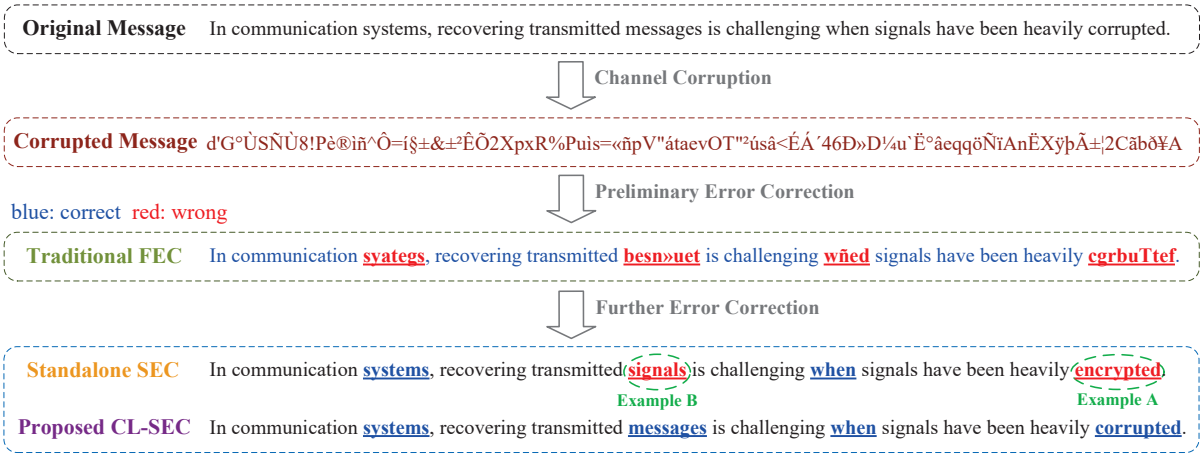

Figure 1: Examples of error correction via different approaches.

- ***Lexical Imprecision:*** Standalone SEC can preserve semantic meaning but fails to maintain lexical accuracy. For example, in Example B, the recovered phrase "*transmitted signals*" aligns semantically with the original message but lexically deviates from the original term "*messages*". Such deviations can lead to critical misunderstandings in contexts requiring verbatim accuracy, such as protocol specifications or technical documentation.

To address these limitations and unlock the full potential of LMs for error correction, we propose Cross-Layer Semantic Error Correction (CL-SEC). It operates after FEC to refine error correction. Unlike standalone SEC, which relies solely on the post-FEC decoded text and aims to recover the semantic meaning, CL-SEC achieves **verbatim text recovery** even under severe corruption by integrating physical-layer FEC probabilities with application-layer semantic probabilities from LMs.

The essence of our approach and main contributions are described as follows:

- **A novel CL-SEC error-correction framework:** Our framework uses words as the basic correction units. Information about word lengths is compressed and embedded in the frame header, allowing the receiver to determine the boundaries of transmitted words. At the receiver, CL-SEC detects potentially erroneous words and generates a candidate set of potential replacements for each one.
- **Dual-layer probability distributions:** CL-SEC leverages two complementary probability distributions to jointly determine the most suitable replacement word from the candidate set: **1)** the *physical-layer probability distribution*, derived from soft log-likelihood ratios (LLRs) produced during channel decoding, and **2)** the *application-layer probability distribution*, extracted from an LM.
- **Cross-layer Bayesian combination:** The complementary probability distributions are combined in product form to calculate cross-layer posterior probabilities for the candidate words. We justify this combination by drawing an analogy with the principle of belief propagation, supplemented by a detailed analytical explanation that highlights precisely where this approach makes an approximation.
- **Extensive experimental validation:** We demonstrate that CL-SEC significantly enhances classical channel decoding by incorporating semantic information, achieving superior verbatim recovery. Our results also show substantial performance gains over methods that rely solely on physical-layer LLRs or application-layer LMs for FEC refinement, with notable improvements across key error-correction metrics, including bit-error rate, word-error rate, and semantic fidelity scores.

2 System Architecture

We consider the transmission of a frame containing a natural language message. Figure 2 shows the block diagram of the communication system with CL-SEC framework, using words as the basic correction units.

At the transmitter, we first remove all punctuation and spaces from the original message $\mathbf{p}$. There are N words left, denoted by $\mathbf{w}_n$, $n \in \mathcal{N} \triangleq \{1, \ldots, N\}$. Let L_n be the number of letters in $\mathbf{w}_n$ (i.e., the word length). Concatenating all words yields a character sequence $\mathbf{s} = (\mathbf{w}_1, \ldots, \mathbf{w}_N)$ of length $L = \sum_{n=1}^{N} L_n$. Information on the word lengths $\{L_n\}_{n\in\mathcal{N}}$ is included in the frame header so that the receiver knows the demarcations of words.[1]

For transmission, each character s_ℓ, $\ell \in \{1, \ldots, L\}$, is converted to its 8-bit ASCII representation. Accordingly, each word $\mathbf{w}_n$ is character-encoded into $\mathbf{u}_n \in \mathbb{F}_2^{K_n}$, where $K_n = 8L_n$ is the number of bits within word $\mathbf{w}_n$ and $\mathbb{F}_2 \triangleq \{0, 1\}$. Concatenation yields bit sequence $\mathbf{u} = (\mathbf{u}_1, \ldots, \mathbf{u}_N) = (u_1, \ldots u_K) \in \mathbb{F}_2^K$ with $K = 8L$. Then, $\mathbf{u}$ is channel-encoded into $\mathbf{c}$ with the code rate R.

[1]We employ canonical Huffman coding to efficiently compress the word-length information – metadata embedded in the header that specifies the lengths of words in the message. Details are provided in Appendix B. Our evaluations indicate that the header overhead is less than 10% for 98.73% of the passages in the corpus. In order to reduce the probability of the header corruption, higher-level FEC could be employed there than the payload. The overhead and corruption of the header could be potentially eliminated if the receiver estimates word lengths instead of relying on the word-length metadata.

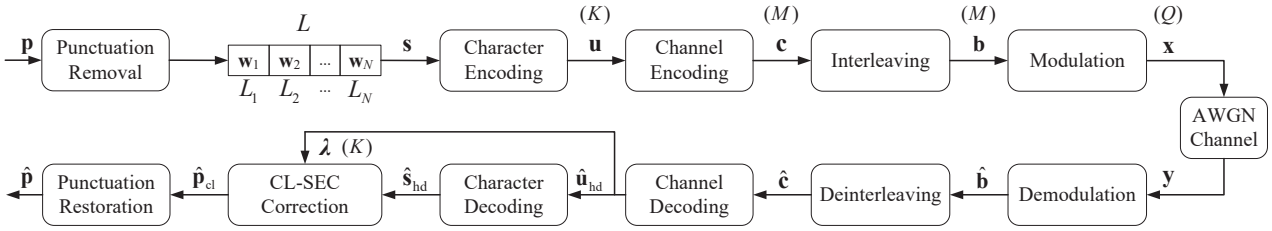

Figure 2: The communication system with CL-SEC framework. The vector dimension is indicated in parentheses.

For concreteness, this paper focuses on convolutional codes; however, our CL-SEC framework is also compatible with other channel codes, such as low-density parity-check (LDPC) codes. For the convolutional code, we assume that each information bit is input into a rate-R encoder to generate $1/R$ bits. The memory length of the convolutional code is ν, and the corresponding constraint length is $\nu+1$. That is, the number of input bits, including the current input bit, that affect a current output bit is $\nu+1$. With ν zero bits flushing the memory data at the end of encoding, the length of the encoded sequence $\mathbf{c}$ is $M=(K+\nu)/R$. Next, $\mathbf{c}$ is randomly permuted into $\mathbf{b}=\pi(\mathbf{c})$, and the interleaved sequence is further mapped to symbols $\mathbf{x}=(x_1,\ldots,x_Q)\in\mathbb{C}^Q$ with the modulation order $2^{M/Q}$. The signal is transmitted over an additive white Gaussian noise (AWGN) channel, and the received signal is $\mathbf{y}=\mathbf{x}+\mathbf{n}$, where $\mathbf{n}\sim\mathcal{CN}(\mathbf{0}_Q,\sigma^2\mathbf{I}_Q)$.

At the receiver, we first perform demodulation to obtain the estimated bits $\hat{\mathbf{b}}\in\mathbb{F}_2^M$. Then, we apply the inverse permutation to get $\hat{\mathbf{c}}=\pi^{-1}(\hat{\mathbf{b}})=(\hat{c}_1,\ldots,\hat{c}_M)$. Next, as shown in Fig. 2, the soft-output channel decoder produces bit-level soft log-likelihood ratios (LLRs) $\boldsymbol{\lambda}=(\lambda_1,\ldots,\lambda_K)\in\mathbb{R}^K$, and further uses hard decision to recover the hard-bit sequence $\hat{\mathbf{u}}_{\text{hd}}\in\mathbb{F}_2^K$ (see Sec. 3.1). This sequence $\hat{\mathbf{u}}_{\text{hd}}$ is further mapped back to the character sequence $\hat{\mathbf{s}}_{\text{hd}}$ of length L via inverse ASCII mapping.

However, there can still be many errors in the post-FEC $\hat{\mathbf{s}}_{\text{hd}}$ under severe corruption. For the enhanced reliability, we propose CL-SEC to further correct the errors in the preliminary FEC results with the help of LLRs (see Sec. 3). Restoring punctuation for the CL-SEC output $\hat{\mathbf{p}}_{\text{cl}}$, we obtain the message recovery $\hat{\mathbf{p}}$.

3 Critical Components in CL-SEC Framework

3.1 Soft Information and Hard Decision

Before delving into CL-SEC error correction, let us first review the classical hard decision (HD)-based correction scheme with the soft-output channel decoder. This scheme is used for the preliminary error correction and LLR generation in our system (Fig. 2). LLRs, as required by CL-SEC, are readily available from channel coding techniques with soft-output channel decoders. This paper employs the convolutional code with a maximum a posteriori (MAP)-based BCJR decoder [1, 14]. However, our framework can also accommodate other soft-output coding schemes, such as LDPC codes with a belief propagation decoder [4, 11].

Recall that $\{\hat{c}_m\}_{m=1}^M$ are the noisy coded bits at the receiver. The decision of each source bit u_k, where $k\in\mathcal{K}\triangleq\{1,\ldots,K\}$, depends merely on the sequence $\hat{\mathbf{c}}_{\text{bit},k}=\big(\hat{c}_{(k-1)/R+1},\ldots,\hat{c}_{(k+\nu)/R}\big)$ with length $(\nu+1)/R$. The BCJR decoder outputs bit-level soft LLR:

$$\lambda_k\triangleq\log\left(\frac{P\big(\hat{\mathbf{c}}_{\text{bit},k}|u_k=0\big)}{P\big(\hat{\mathbf{c}}_{\text{bit},k}|u_k=1\big)}\right)\overset{(a)}{=}\log\left(\frac{P\big(u_k=0|\hat{\mathbf{c}}_{\text{bit},k}\big)}{P\big(u_k=1|\hat{\mathbf{c}}_{\text{bit},k}\big)}\right),\ k\in\mathcal{K},\quad(1)$$

where (a) follows from the common assumption that bits 0 and 1 are equally likely, i.e., $P(u_k=0)=P(u_k=1)=1/2$. The bit-wise posterior probabilities $P\big(u_k=0|\hat{\mathbf{c}}_{\text{bit},k}\big)$, where $u_k\in\mathbb{F}_2$, can be obtained from each λ_k. HD then yields the post-FEC hard bit purely produced by the physical layer:

$$\hat{u}_{\text{hd},k}=\arg\max_{u_k\in\mathbb{F}_2}P\big(u_k|\hat{\mathbf{c}}_{\text{bit},k}\big),\ k\in\mathcal{K}.\quad(2)$$

Mapping the bit sequence $\hat{\mathbf{u}}_{\text{hd}}=\big(\hat{u}_{\text{hd},1},\ldots,\hat{u}_{\text{hd},K}\big)$ to characters and segmenting the resulting character sequence into words using the word-length metadata yield $\hat{\mathbf{s}}_{\text{hd}}=\big(\hat{\mathbf{w}}_{\text{hd},1},\ldots,\hat{\mathbf{w}}_{\text{hd},N}\big)$, where $\hat{\mathbf{w}}_{\text{hd},n}$'s are the HD-based word estimates. We next propose CL-SEC to further refine the preliminary FEC results $\{\hat{\mathbf{w}}_n\}_{n=1}^N$ at the word level aided by LLRs.

3.2 CL-SEC Overview

CL-SEC performs error correction with words as the basic correction units. Specifically, we **1)** first detect erroneous words in $\hat{\mathbf{s}}_{\text{hd}}$, and for each erroneous word, construct a list of candidate replacement words (see Sec. 3.3) – these candidates have a word length as indicated by the metadata in the header; **2a)** construct a physical-layer probability distribution among the candidates based on the LLRs of the corresponding bits, representing signal-level evidence (see Sec. 3.4); and, in parallel, **2b)** determine an application-layer probability distribution among the candidates, as assessed by the LM given the context provided by surrounding words in the message (see Sec. 3.5); and **3)** Bayesian-combine these two complementary distributions in product form to obtain a cross-layer posterior distribution among the candidates, from which we choose the candidate with the highest probability to correct for the erroneous word (see Sec. 3.6).

In Step 2b we mask erroneous words and insert spaces to construct the message $\hat{\mathbf{p}}_{\text{m}}$, which is input to a masked language model (MLM) for mask prediction. After CL-SEC, we replace the masked words in $\hat{\mathbf{p}}_{\text{m}}$ with CL-SEC correction results and obtain $\hat{\mathbf{p}}_{\text{cl}}$. Finally, we restore punctuation for $\hat{\mathbf{p}}_{\text{cl}}$ based on context (see Sec. 3.7), producing the final message recovery $\hat{\mathbf{p}}$.

3.3 Error Detection and Correction Candidates

The HD word estimates $\{\hat{\mathbf{w}}_{\text{hd},n}\}_{n\in\mathcal{N}}$ may contain garbled letters, resulting in erroneous words. We detect word-level errors as follows. Let $\mathcal{W}$ denote the vocabulary set acquired from the tokenizer

of an LM, with cardinality $S = |\mathcal{W}|$. Detailed Setups of the LM will be specified later in Sec. 3.5. For each $n \in \mathcal{N}$, if $\hat{\mathbf{w}}_{\mathrm{hd},n} \in \mathcal{W}$, we regard $\hat{\mathbf{w}}_{\mathrm{hd},n}$ as correct; otherwise, $\hat{\mathbf{w}}_{\mathrm{hd},n}$ is deemed erroneous and to be corrected. Define the index set of erroneous words as $\mathcal{N}_{\mathrm{e}} \triangleq \{n|\hat{\mathbf{w}}_{\mathrm{hd},n} \notin \mathcal{W}\} \subseteq \mathcal{N}$.

CL-SEC leverages information from both the physical layer and the application layer to correct the erroneous post-HD words. With the word-length metadata, for each $n \in \mathcal{N}_{\mathrm{e}}$ we form a candidate-correction set $\mathcal{W}_n \subset \mathcal{W}$ consisting of all words in $\mathcal{W}$ with word length L_n and the cardinality is $S_n = |\mathcal{W}_n|$. We select the candidate in $\mathcal{W}_n$ with the highest probability to replace the erroneous word.[2] The computation of the probability distribution among the candidates in $\mathcal{W}_n$ is described in the following Sections 3.4–3.6.

3.4 The Word-Level Physical-Layer Probability Distribution

This section presents the word-level physical-layer probability distribution used in CL-SEC. For each candidate word $\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n$, $s \in \mathcal{S}_n \triangleq \{1, \ldots, S_n\}$, we convert the candidate word to its ASCII bit representation $\tilde{\mathbf{u}}_{n,s} = (\tilde{u}_{n,s,1}, \ldots, \tilde{u}_{n,s,K_n}) \in \mathbb{F}_2^{K_n}$, where K_n is the number of source bits within word $\mathbf{w}_n$. Define $\tilde{K}_n = \sum_{j=1}^{n-1} K_j$ as the total number of bits precede those whtin $\mathbf{w}_n$. Further define sequence

$$\hat{\mathbf{c}}_n \triangleq \left(\hat{c}_{\tilde{K}_n/R+1}, \ldots, \hat{c}_{(\tilde{K}_{n+1}+\nu)/R}\right), \tag{3}$$

and its length is $(K_n + \nu)/R$. The encoded-bit sequence $\hat{\mathbf{c}}_n$ determines the physical-layer decision of the word $\mathbf{w}_n$.

The physical-layer posterior probability for candidate $\tilde{\mathbf{w}}_{n,s}$ is found from

$$P(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n) = \prod_{k=1}^{K_n} P\left(u_{\tilde{K}_n+k} = \tilde{u}_{n,s,k} \middle| \hat{\mathbf{c}}_{\mathrm{bit},\tilde{K}_n+k}\right),\ n \in \mathcal{N}_{\mathrm{e}}, s \in \mathcal{S}_n, \tag{4}$$

where we range k within the individual bit sequence associated with word $\mathbf{w}_n$. Unlike the classical bit-level channel decoding used for bit decision (see Sec. 3.1), we refer to (4) as the *word-level channel decoding*: it is tailored to determine the posterior probability $P(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n)$ for word decision using LLRs produced from classical soft channel decoding. Accordingly, we refer to LLRs used in (4), i.e., $\lambda_{\tilde{K}_n+1}, \ldots, \lambda_{\tilde{K}_{n+1}}$, $n \in \mathcal{N}_{\mathrm{e}}$, as *word-level LLRs (WL-LLRs)*. Collecting the probabilities of all S_n candidates yields a word-level physical-layer distribution for word $\mathbf{w}_n$, denoted by $\mathbf{d}_{\mathrm{ph},n}(\hat{\mathbf{c}}_n) \propto \left(P(\tilde{\mathbf{w}}_{n,1}|\hat{\mathbf{c}}_n), \ldots, P(\tilde{\mathbf{w}}_{n,S_n}|\hat{\mathbf{c}}_n)\right)$.[3]

Simple WL-LLR Benchmarking Scheme: Before discussing the application-layer probability distribution used in CL-SEC, let us pause to present a simple correction scheme based on WL-LLRs. We treat this WL-LLR scheme as one of the benchmarks for our CL-SEC. Comparison of their performance is provided in Sec. 4. WL-LLR scheme is developed purely using the distribution $\mathbf{d}_{\mathrm{ph},n}(\hat{\mathbf{c}}_n)$ to determine the most likely word with length L_n for FEC refinement:

$$\hat{\mathbf{w}}_{\mathrm{ph},n} = \underset{\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n}{\arg\max}\, P(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n),\ n \in \mathcal{N}_{\mathrm{e}}. \tag{5}$$

Unlike the FEC based on the bit-level LLRs (see Sec. 3.1), WL-LLR scheme operates at the word level. Notably, the simple WL-LLR scheme uses the vocabulary of an LM to impose word-level constraints without leveraging the ability of the LM to infer the likelihood of the word based on the surrounding words.[4] In the following, we proceed to discuss the construction of the application-layer distribution in CL-SEC.

3.5 Application-Layer Probability Distribution

Recent advances in MLM enable correction of erroneous words using the application-layer semantic context captured by pretrained models (e.g., BART [5] and mmBERT [7]). In MLM pretraining, a subset of input tokens is randomly masked, and the model learns to infer the masked tokens from the surrounding semantic context – a compositional blank-filling ability found in human communication. For example, given the sentence "There is a beach with palm [MASK] and clear blue water," a pretrained MLM may predict the missing word as, for example, "trees".

In the HD output $\hat{\mathbf{s}}_{\mathrm{hd}}$, for each $n \in \mathcal{N}_{\mathrm{e}}$, replace the n-th word that is erroneous with a mask token (e.g., [MASK]). Further, insert spaces between words according to the word-length metadata to facilitate the semantic understanding of the MLM.[5] These procedures yield a message with masks and spaces, denoted by $\hat{\mathbf{p}}_{\mathrm{m}}$. The tokenizer of an MLM converts $\hat{\mathbf{p}}_{\mathrm{m}}$ into a token-index sequence

$$\hat{\mathbf{t}} = f_{\mathrm{tok}}(\hat{\mathbf{p}}_{\mathrm{m}}) = (\hat{t}_1, \ldots, \hat{t}_I), \tag{6}$$

where $\hat{t}_i$ is the vocabulary index (i.e., token ID) of the token at position i, with $\hat{t}_i \in \{1, \ldots, S\}$, $i = 1, \ldots, I$ and $S = |\mathcal{W}|$. Each token is mapped to a high-dimensional embedding that captures semantic information. At token position i, the MLM produces a posterior probability distribution from embeddings over the vocabulary set. This distribution is denoted by $P(t_i'|\check{\boldsymbol{\xi}}_i, \mathrm{MLM})$, where $t_i' \in \{1, \ldots, S\}$ is the potential output token ID, $\check{\boldsymbol{\xi}}_i$ represents the surrounding context to token $\hat{t}_i$, and "MLM" means the availability of a specifically given MLM.

Since each erroneous word is masked using the mask token in the MLM input, we locate these mask tokens and obtain the associated MLM outputs for probability extraction. Specifically, for each masked-word position $n \in \mathcal{N}_{\mathrm{e}}$, we extract the probabilities of the candidate words $\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n \subset \mathcal{W}$ from the MLM output over the entire vocabulary $\mathcal{W}$, yielding selected posterior probabilities $P(\tilde{\mathbf{w}}_{n,s}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM})$, $s \in \mathcal{S}_n$, with the surrounding context $\hat{\boldsymbol{\xi}}_n$ to word n (words surrounding word n), and with the availability of a specific MLM. Collecting these S_n probabilities obtains the application-layer distribution $\mathbf{d}_{\mathrm{ap},n}(\hat{\boldsymbol{\xi}}_n, \mathrm{MLM}) \propto \left(P(\tilde{\mathbf{w}}_{n,1}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM}), \ldots, P(\tilde{\mathbf{w}}_{n,S_n}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM})\right)$ among the candidate

[2] Note that the vocabulary used for word-error detection could be different from the vocabulary used for word-error correction. While the LM-based word-error correction relies on the vocabulary of an LM, we could use (or construct) a larger vocabulary (not necessarily from an LM) for more accurate error detection.

[3] For numerical stability, the probabilities should be normalized via a positive scaling constant in practical implementations. The same is true for the other probability distributions.

[4] Like the earlier discussion on word-error detection, we could use any vocabulary (not necessarily from an LM) to determine word replacements in WL-LLR scheme.

[5] Note that words at this stage can still be heavily corrupted, making punctuation restoration challenging. Fortunately, the MLM can still make effective predictions even when punctuation is absent from the input, as demonstrated by our simulations (see Sec. 4). We will restore punctuation based on context after accurate word corrections (see Sec. 3.7).

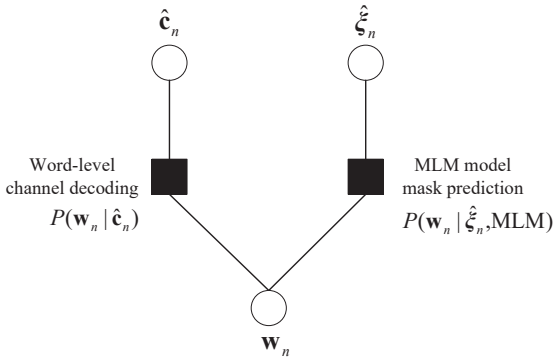

Figure 3: A factor graph for CL-SEC, $n \in \mathcal{N}_e$. The empty circles represent variable nodes, while the filled squares represent function nodes.

corrections. With dual-layer distributions $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$ above and $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ discussed in Sec. 3.4, we next Bayesian-combine them for CL-SEC (see Sec. 3.6).

Standalone SEC (Simple MLM) Benchmarking Schemes: We next discuss standalone SEC (i.e., simple MLM) schemes, serving as benchmarks for our CL-SEC. Standalone SEC can be categorized into two types, depending on whether word-length information is available. For the type without such information (e.g., the "Standalone SEC" demonstrated in the fourth box of Fig. 1), the word correction may have an incorrect word length (e.g., the word "signals" in Fig. 1).

To benchmark CL-SEC in a fair manner (in Sec. 4), we consider a more refined standalone SEC with word-length information. The post-FEC correction of this simple MLM scheme is found purely from $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$ by

$$\hat{\mathbf{w}}_{\text{ap},n} = \underset{\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n}{\arg\max}\, P\big(\tilde{\mathbf{w}}_{n,s} | \hat{\boldsymbol{\xi}}_n, \text{MLM}\big),\ n \in \mathcal{N}_e. \tag{7}$$

Like WL-LLR, the MLM scheme above enforces word-level constraints through the LM's vocabulary. Unlike WL-LLR, however, it also leverages the semantic prediction ability of the model.

3.6 Cross-Layer Word Correction

From the discussions in Sections 3.4 and 3.5, for each erroneous word $n \in \mathcal{N}_e$, we could obtain two posterior probability distributions: the physical-layer distribution $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ constructed from LLRs, and the application-layer distribution $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$ as assessed by the MLM. We next analyze the dependencies between these two distributions. Then we propose to Bayesian-combine these distributions to determine a cross-layer posterior probability distribution for cross-layer word correction, co-determined by the dual-layer information.

We employ the factor graph and the principle of belief propagation (BP) to justify the Bayesian combination adopted in CL-SEC. We do not intend this to be a rigorous mathematical derivation following the BP construct, as the process by which an MLM infers a masked word from the surrounding context does not rely on the same equations as a typical BP algorithm. However, the concept here is analogous to BP in that the masked word is inferred based on the "beliefs" derived from the surrounding words.

A factor graph for CL-SEC is shown in Fig. 3. The key point we aim to highlight is that for each erroneous word indexed by $n \in \mathcal{N}_e$, the two information sources $\hat{\mathbf{c}}_n$ and $\hat{\boldsymbol{\xi}}_n$ leveraged in word-level channel decoding and MLM mask prediction are only **weakly dependent**. Specifically, the two conditional distributions $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ and $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$ can be approximately treated as independent (i.e., their beliefs are approximately from independent sources $\hat{\mathbf{c}}_n$ and $\hat{\boldsymbol{\xi}}_n$), and thus these two distributions can be Bayesian-combined in **product form** [6]. We analyze this weak dependence relation in the following:

With reference to (3), $\hat{\mathbf{c}}_n$ consists of coded bits $\hat{c}_{\tilde{K}_n/R+1}, \ldots, \hat{c}_{(\tilde{K}_{n+1}+\nu)/R}$. These bits are related to the source bits within the masked word. Meanwhile, $\hat{\boldsymbol{\xi}}_n$ is derived from all the words surrounding the mask and is thereby associated with two parts of coded bits: $\hat{c}_1, \ldots, \hat{c}_{\tilde{K}_n/R+1}, \ldots, \hat{c}_{(\tilde{K}_n+\nu)/R}$ and $\hat{c}_{\tilde{K}_{n+1}/R+1}, \ldots, \hat{c}_{(\tilde{K}_{n+1}+\nu)/R}, \ldots, \hat{c}_M$. The first part is related to certain adjacent source bits before those within the mask, while the second part is associated with certain adjacent source bits after the mask. There are very few double dips on the used information: merely the left boundary bits $\hat{c}_{\tilde{K}_n/R+1}, \ldots, \hat{c}_{(\tilde{K}_n+\nu)/R}$ and right boundary bits $\hat{c}_{\tilde{K}_{n+1}/R+1}, \ldots, \hat{c}_{(\tilde{K}_{n+1}+\nu)/R}$ in $\hat{\mathbf{c}}_n$ ($2\nu/R$ bits altogether). Meanwhile, $\hat{\mathbf{c}}_n$ in $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ typically contains many bits associated with characters within a masked word and the $2\nu/R$ boundary bits are just a small portion of $\hat{\mathbf{c}}_n$ on which $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ depends. Further, for $\hat{\boldsymbol{\xi}}_n$ in $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$, the non-boundary bits in $\hat{\mathbf{c}}_n$ do not contribute to $\hat{\boldsymbol{\xi}}_n$, and the $2\nu/R$ boundary bits in $\hat{\mathbf{c}}_n$ contribute little because $\hat{\boldsymbol{\xi}}_n$ is mainly derived from other words than the masked word. For concreteness, let us consider $\nu = 2$ and $R = 1/2$, as employed in our simulations (see Sec. 4), only $2\nu/R = 8$ boundary coded bits (or equivalently, 1/2 of a source character) are doubly used, once in $\mathbf{d}_{\text{ph},n}(\hat{\mathbf{c}}_n)$ and once in $\mathbf{d}_{\text{ap},n}(\hat{\boldsymbol{\xi}}_n, \text{MLM})$.

In conclusion, the weak dependency is attributed to two reasons: First, the attention span of the LM is far wider than the constraint length of the convolutional code. Second, since the information within each erroneous word has already been utilized for physical-layer word-level channel decoding, we intentionally remove such information via mask replacement in the MLM implementation to minimize double dip.

With the weak dependence relation as justified above, we approximate the factor graph (Fig. 3) as a loop-free graph, where it is straightforward to compute the exact marginal of $\mathbf{w}_n$ without the need of iterations. Following the belief update rule [4], the cross-layer marginal of $\mathbf{w}_n$, $n \in \mathcal{N}_e$, is computed from

$$P\big(\tilde{\mathbf{w}}_{n,s} | \hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \text{MLM}\big) \propto P\big(\tilde{\mathbf{w}}_{n,s} | \hat{\mathbf{c}}_n\big) \cdot P\big(\tilde{\mathbf{w}}_{n,s} | \hat{\boldsymbol{\xi}}_n, \text{MLM}\big),\ s \in \mathcal{S}_n. \tag{8}$$

That is, the marginal is obtained from the product of two beliefs flowing down from the two function nodes. With (8), we obtain the cross-layer posterior probability through the Bayesian combination of two single-layer posterior probabilities in product form.

[6] We note that the classical BP algorithm also employs a Bayesian combination of beliefs in product form, even when the corresponding factor graph contains loops (i.e., the beliefs being combined are from dependent sources because the belief of a source may propagate back though a loop to the point of combination). Nevertheless, this product-form approximation proves to be highly effective for many problems, such as LDPC decoding.

Prompt:

You are a professional natural language editor specializing in punctuation recovery. Your task:
- Read the following unpunctuated text carefully.
- Restore appropriate punctuation marks (commas, periods, question marks, exclamation marks, etc.) based on context, grammar, and natural readability.
- Only output the final text with proper punctuations.
- Do not include any explanations, notes, or formatting outside the text itself.

Text without punctuation:
(Insert here the text without punctuation)

Now, only output the text restored with correct punctuations.

Example Input:

How many spiders are engaged in this world on our behalf One authority on spiders made a census of the spiders in grass field in the south of England and he estimated that there were more than two million in one acre That is something like six million spiders of different kinds on a football pitch These creatures are busy for at least half the year in killing insects It is impossible to make more than a rough guess at how many they kill but they are hungry creatures not content with only three meals a day It has been estimated that the weight of all the insects destroyed by spiders in Britain in one year would be greater than the total weight of all the human beings in the country

Example Output:

How many spiders are engaged in this world on our behalf? One authority on spiders made a census of the spiders in grass field in the south of England, and he estimated that there were more than two million in one acre. That is something like six million spiders of different kinds on a football pitch. These creatures are busy for at least half the year in killing insects. It is impossible to make more than a rough guess at how many they kill, but they are hungry creatures, not content with only three meals a day. It has been estimated that the weight of all the insects destroyed by spiders in Britain in one year would be greater than the total weight of all the human beings in the country.

Figure 4: The prompt, example input, and example output of punctuation restoration.

To better clarify the rationale underlying our CL-SEC formulation, a justification for the product form in (8) following detailed probability derivations is provided in Appendix A.

Collecting S_n probabilities associated with S_n candidate words for each mask, we construct a cross-layer posterior probability distribution for each word $\mathbf{w}_n$ as $\mathbf{d}_{\mathrm{cl},n}(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}) \propto \mathbf{d}_{\mathrm{ph},n}(\hat{\mathbf{c}}_n) \circledast \mathbf{d}_{\mathrm{ap},n}(\hat{\boldsymbol{\xi}}_n, \mathrm{MLM})$, where $\circledast$ denotes the Hadamard (element-wise) product. The CL-SEC decoding result is found by the maximum cross-layer posterior probability in $\mathbf{d}_{\mathrm{cl},n}(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM})$ as

$$\hat{\mathbf{w}}_{\mathrm{cl},n} = \underset{\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n}{\arg\max}\, P\big(\tilde{\mathbf{w}}_{n,s} | \hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big),\ n \in \mathcal{N}_{\mathrm{e}}. \tag{9}$$

CL-SEC enforces word-level linguistic constraints through LM's vocabulary and the word-length metadata. Note that a vocabulary may have several versions of a word, such as lowercase, uppercase and capitalized versions. CL-SEC leverages information from both the physical and application layers to jointly determine the most suitable replacement from the vocabulary to recover the transmitted message verbatim. Replacing each mask in $\hat{\mathbf{p}}_{\mathrm{m}}$ with $\{\hat{\mathbf{w}}_{\mathrm{cl},n}\}_{n\in\mathcal{N}_{\mathrm{e}}}$ accordingly, we obtain the message estimate $\hat{\mathbf{p}}_{\mathrm{cl}}$.

3.7 Punctuation Restoration (PR)

Compared with the original message $\mathbf{p}$, the estimated message $\hat{\mathbf{p}}_{\mathrm{cl}}$ lacks punctuation, degrading semantic fidelity and readability. To address this issue, we prompt Qwen3 [15] to restore punctuation for the CL-SEC output using contextual cues and grammatical constraints, as shown in Fig. 4, to arrive at the final estimated message $\hat{\mathbf{p}}$.

Algorithm 1 The CL-SEC framework.

Input: HD results $\{\hat{\mathbf{w}}_{\mathrm{hd},n}\}_{n\in\mathcal{N}}$, LLRs $\{\lambda_k\}_{k\in\mathcal{K}}$, word lengths $\{L_n\}_{n\in\mathcal{N}}$, and vocabulary $\mathcal{W}$.

Implementation:

1: Determine indices of erroneous words $\mathcal{N}_{\mathrm{e}} = \{n \mid \hat{\mathbf{w}}_{\mathrm{hd},n} \notin \mathcal{W}\}$.
2: **for** each $n \in \mathcal{N}_{\mathrm{e}}$ **do**
3: Determine candidates $\mathcal{W}_n$ using the word length L_n.
4: Obtain physical-layer distribution $\mathbf{d}_{\mathrm{ph},n}$ from LLRs via (4).
5: Obtain the application-layer distribution $\mathbf{d}_{\mathrm{ap},n}$ from context as assessed by MLM.
6: Compute the cross-layer distribution $\mathbf{d}_{\mathrm{cl},n}$ using (8).
7: Get the cross-layer word correction using (9).
8: **end for**
9: Use CL-SEC corrections to obtain the message estimate $\hat{\mathbf{p}}_{\mathrm{cl}}$.
10: Instruct Qwen3 to restore punctuation for $\hat{\mathbf{p}}_{\mathrm{cl}}$ and get the final message estimate $\hat{\mathbf{p}}$.

Output: Recovered message $\hat{\mathbf{p}}$.

Table 1: Statistics of the corpus with 50 passages.

	Minimum	Maximum	Mean	Std dev
Number of words	85	165	120.0	23.3
Number of letters	357	684	494.8	101.3

The proposed CL-SEC framework is outlined in Algorithm 1. Notably, after the preliminary FEC, Steps 4 and 5 can be implemented in parallel to obtain dual-layer probability distributions; then these complementary distributions are Bayesian-combined to directly refine FEC results, without any intermediate correction steps.

4 Simulation Results

4.1 Experimental Setups

Models: For masked-word prediction, we employ two pretrained models for experimental validations – BART [5] and mmBERT [7] – and evaluate their error-correction performance individually. BART is a classical MLM while mmBERT is a state-of-the-art MLM. We determine the vocabulary set from the tokenizer of BART and mmBERT, denoted by $\mathcal{W}_{\mathrm{BART}}$ and $\mathcal{W}_{\mathrm{mmBERT}}$, respectively. The set $\mathcal{W}_{\mathrm{mmBERT}}$ is larger than $\mathcal{W}_{\mathrm{BART}}$. We use these two sets individually for word correction. We use Qwen3-8B [15] to restore punctuation via prompt engineering. To reduce latency, we disable the reasoning capability of Qwen3.

Dataset: We use 50 passages[7] from English textbooks as original messages to evaluate CL-SEC and its benchmarks. As the FEC refinement methods (to be discussed soon) rely on the vocabulary set for error correction, we preprocess the passages to ensure all words are included in both the vocabulary sets $\mathcal{W}_{\mathrm{BART}}$ and $\mathcal{W}_{\mathrm{mmBERT}}$. Table 1 shows the statistics of these 50 passages. We simulate the transmission of each passage under varying signal-to-noise ratio (SNR). This transmission procedure is repeated over 10 Monte-Carlo trials with random noises for each passage, yielding 500 received corrupted passages at every tested SNR to ensure statistical significance.

[7]Data open available at: https://github.com/Yirun719/CL-SEC

System Settings: We consider a convolutional code with memory length $\nu = 2$ and code rate $R = 1/2$, and leverage the MAP-based BCJR decoder [1, 14] to produce soft LLRs. We apply a random interleaver to permute the bit sequence and use quadrature phase-shift keying (QPSK). The SNR is defined as SNR $= A^2/\sigma^2$, where $A^2 = \mathbb{E}[|x_q|^2]$, $q = 1, \ldots, Q$ is the average signal power and σ^2 is the noise power.[8]

Evaluation Metrics: Two metrics we evaluate in Sec. 4.2 are bit-error rate (BER) and word-error rate (WER). For WER, a word is deemed correct only if all letters in it are correct. In addition, we investigate the semantic fidelity in Sec. 4.3 using BERTScore [16] and ROUGE-L [6]. BERTScore measures semantic similarity by aligning tokens in the original and recovered messages. ROUGE-L captures semantic content preservation by quantifying content overlap based on the longest common subsequence (LCS) between the two messages. Both BERTScore and ROUGE-L are reported on a [0, 100] scale.

Investigated Schemes: We present and compare the performance of five schemes:

- **Scheme 1: BCJR-based HD (Sec. 3.1).** This scheme applies HD at the bit level on the bit-level soft LLRs produced by the classical BCJR decoder (see (1) and (2)). This scheme is essentially the classical channel decoding method. This classical scheme is labelled by "BCJR".
- **Scheme 2: WL-LLR scheme (Sec. 3.4).** This scheme employs one of the two vocabulary sets, $\mathcal{W}_{\text{BART}}$ or $\mathcal{W}_{\text{mmBERT}}$, to obtain the word probabilities from word-level LLRs (see (4)). HD (see (5)) is then made to correct erroneous words in the BCJR results, without using semantic abilities of the application-layer LM. This scheme is labelled by "WL-LLR ($\mathcal{W}_{\text{BART}}$)" and "WL-LLR ($\mathcal{W}_{\text{mmBERT}}$)" according to whether $\mathcal{W}_{\text{BART}}$ or $\mathcal{W}_{\text{mmBERT}}$ is employed.
- **Scheme 3: MLM scheme (Sec. 3.5).** This scheme applies BART and mmBERT separately for MLM correction to BCJR results at the word level (see (7)), without using the LLRs produced by the physical layer. This scheme is labelled by "BART" and "mmBERT" according to which MLM is used.
- **Scheme 4: CL-SEC (WL-LLRs + MLM, Sec. 3.6).** This proposed scheme combine the WL-LLRs and the MLM in Schemes 2 and 3 to correct erroneous words in the BCJR results (see (8) and (9)). Both BART- and mmBERT-based CL-SEC are investigated. This scheme is labeled by "CL-SEC (BART)" and "CL-SEC (mmBERT)" accordingly.
- **Scheme 5: CL-SEC + PR (Sec. 3.7).** Building on our CL-SEC results (Scheme 4), this scheme further uses Qwen3 for punctuation restoration (PR). Both PR results for BART- and mmBERT-based CL-SEC corrections are reported, labeled as "CL-SEC(BART) + PR" and "CL-SEC(mmBERT) + PR", respectively.

[8]We have also performed evaluations under 16PSK modulation. We find that the reliability performances of QPSK scheme operated at a certain SNR can be similarly achieved by 16PSK scheme at a higher SNR. Due to space limitations, we only report the QPSK results in this paper.

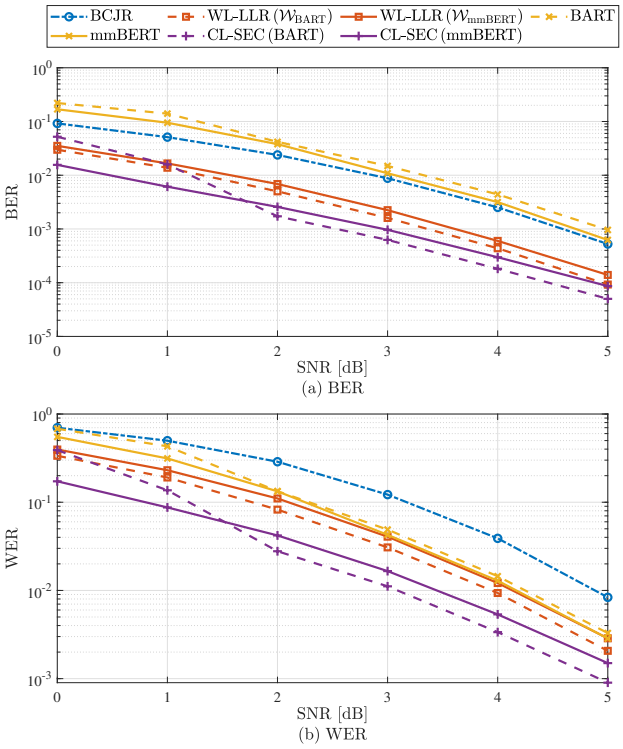

Figure 5: Performance comparison with varying SNR: (a) BER and (b) WER.

4.2 BER and WER Performances

This section evaluates the recovery BER and WER results of the transmitted message. As punctuation and spaces are not considered in BER and WER calculations, only Schemes 1–4 are evaluated in this section. Scheme 5 will be evaluated later in Sec. 4.3, in which we assess the semantic fidelity of all five schemes. For BER calculation, the character-level results of Schemes 2–4 are converted to the bit-level representations.

The BER comparison is shown in Fig. 5(a). The classical BER results of the BCJR decoding scheme is shown here. This pure physical-layer decoding performance serves as an important benchmark for other schemes that, in addition to physical-layer signal information, also inject different degrees of semantic understanding into the decoding process. Given its importance as a benchmark, we have verified the BCJR-based HD scheme, when applied to the test messages in our experiments, achieves a BER consistent with the well-known BER of the BCJR scheme [12] when applied to random bits.

The key point we aim to highlight in this section is that incorporating semantic error correction (i.e., our CL-SEC) can significantly improve the classical BER performance of BCJR, achieving superior verbatim recovery.

Let us examine the BER results in Fig. 5(a). First, we find that mmBERT-based CL-SEC scheme consistently improves the BER of the $\mathcal{W}_{\text{mmBERT}}$-based WL-LLR baseline by incorporating semantic context. The performance advantage of BART-based CL-SEC over

$\mathcal{W}_{\text{BART}}$-based WL-LLR is also observed at high SNRs, but at low SNR (e.g., 0 dB), the CL-SEC scheme performs a little worse than WL-LLR. At low SNR, the number of masked words is high. Thus, for a given mask, the useful context information available from its surrounding is also reduced due to the large number of masks in the context. BART is not as good as mmBERT in inducing information on the masked word when the surrounding context is poor.

Second, we note that the pure MLM-based correction using either BART or mmBERT (without using physical-layer LLRs) results in a worse BER than the classical BCJR decoding. This outcome is not surprising, as the context-driven MLM may revise the corrupted post-BCJR word into a semantically similar but lexically different word from the ground truth. That is, when the BCJR-output word contains only a few incorrect letters (or bits), the MLM may replace it with a word that differs from the original, potentially introducing even more letter-level (or bit-level) errors.

Third, both WL-LLR schemes consistently outperform pure BCJR on BER, demonstrating the effectiveness of the post-BCJR WL-LLR correction at the word level. We also observe that $\mathcal{W}_{\text{BART}}$-based WL-LLR scheme performs better than the $\mathcal{W}_{\text{mmBERT}}$-based WL-LLR. Since we ensure that all words in the original messages are included in both $\mathcal{W}_{\text{BART}}$ and $\mathcal{W}_{\text{mmBERT}}$, the larger $\mathcal{W}_{\text{mmBERT}}$ provides more correction candidates (beyond those can be in the original messages), diluting the sharpness of the correction process.

Figure 5(b) shows the WER results. Overall, the results here are consistent with the BER results in Fig. 5(a). Our CL-SEC schemes achieve better word-correction performance than the corresponding WL-LLR and MLM schemes. A key difference from Fig. 5(a) is that MLM-based corrections to the BCJR output are effective at the word level, making BCJR operated purely at the bit level the worst scheme in terms of WER. Indeed, BCJR struggles to recover the exact transmitted words, leaving many corrupted words to be corrected. With contextual information, the MLM can successfully repair a portion of these words.

4.3 BERTScore and ROUGE-L Performances

In this section, we evaluate the semantic fidelity of all five schemes using BERTScore and ROUGE-L (see Sec. 4.1). For each metric, we feed the original message (with punctuation and spaces) and its recovered version into the corresponding scoring function. For Schemes 1–4, we input the recovered message that has spaces between words but without punctuation. For Scheme 5, we input the PR results built on CL-SEC (Scheme 4) as the recovered version.[9]

The BERTScore and ROUGE-L comparisons are shown in Fig. 6. We observe that the results under these two metrics are consistent. First, by combining the WL-LLRs and the MLM, our CL-SEC method can surpass the semantic fidelities of the corresponding WL-LLR and MLM schemes alone. Second, applying Qwen3-based PR for the CL-SEC output yields the best semantic fidelity. Third, WL-LLR and MLM schemes achieve higher semantic scores than the raw bit-level BCJR baseline, demonstrating their correction effectiveness at the semantic level.

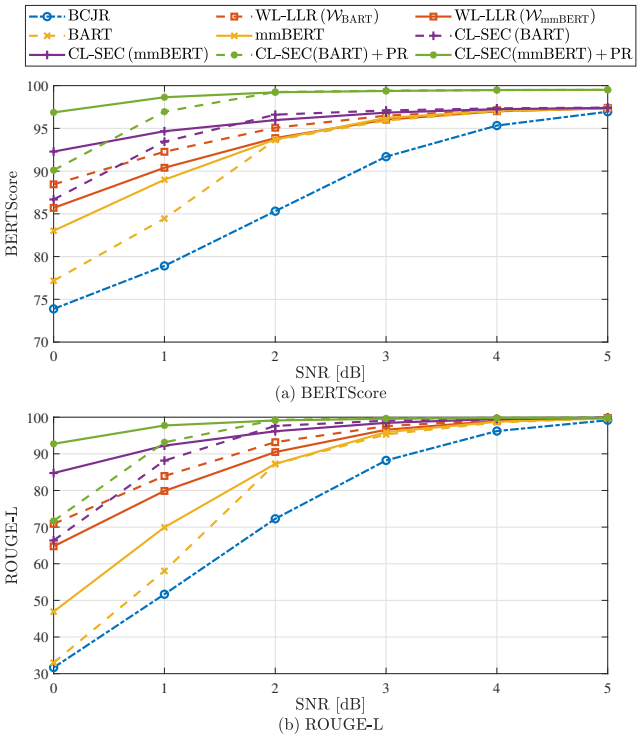

Figure 6: Performance comparison with varying SNR: (a) BERTScore and (b) ROUGE-L.

5 Related Work

The rapid progress of LMs offers a new frontier for error correction techniques. A recent work [3] proposed to use LM trained with massive materials to perform SEC. As the standalone SEC schemes described in Sec. 3.5, the standalone SEC scheme [3] serves as a complementary error-correction mechanism at the application layer to the traditional physical-layer FEC. It is applied after channel decoding at the physical layer followed by masked (and unmasked) sequence identification.[10] This work reported that the additional LM-based SEC substantially improves both the semantic fidelity and the block-error rate (BLER)[11].

[9] Note that Schemes 1–4 compete fairly without punctuation inserted. The performance advantage of CL-SEC (Scheme 4) over Schemes 1–3 can be found in Fig. 6. For the recovery with punctuation, we report PR only built on CL-SEC. Applying PR to Schemes 1–3 would propagate their residual word errors into the PR stage and thus would yield inferior PR results compared with CL-SEC + PR, making the additional curves largely uninformative.

[10] To refine FEC, reference [3] focuses on the sequence-level correction, whereas our work considers word-level correction. This mismatch in problem formulation makes a direct performance comparison challenging. However, we note that the standalone SEC scheme [3] relies solely on a SEC model for error correction, without leveraging metadata. Within our framework, such a standalone SEC performs worse than the refined MLM scheme with metadata (i.e., word-length information). Furthermore, our CL-SEC is significantly superior to the refined MLM scheme, as shown in Figs. 5 and 6. The use of metadata is also not considered in purely LM-based SEC schemes in [2, 9]. Therefore, prior standalone SEC schemes are inferior to CL-SEC within our framework.

[11] Note that BLER is a metric for verbatim recovery – a character block (sequence) is deemed wrong even with a single incorrect character. Prior standalone SEC schemes [2, 3, 9] tend to be inferior on verbatim-recovery metrics such as BLER since they purely rely on SEC models that only aim to preserve the semantic meaning.

Similarly, another study [2] employed a pretrained LM model for SEC at the receiver in understanding-level semantic communication (ULSC) systems, significantly reducing the semantic loss. In addition, the work [9] investigated a token-domain multiple access (ToDMA) framework and proposed a SEC approach to mitigate token collisions in non-orthogonal transmission.

Overall, prior standalone SEC schemes operate purely at the application layer and thus only have access to application-layer information after FEC, overlooking the rich information available at the physical layer for post-FEC error correction, such as the highly valuable bit-level soft LLRs. Relying on a single source of information – the semantic information, the prior standalone SEC schemes remain sub-optimal, suffering from *semantic inaccuracy* and *lexical imprecision* (see Sec. 1).

To address the limitations above, this work leverages both the physical- and application-layer information to refine error corrections by FEC. The proposed Cross-Layer SEC (`CL-SEC`) framework aims to recover the transmitted message **verbatim**, aided by the semantic and linguistic capabilities of LMs for precise reconstruction.

In many semantic communication systems, the primary objective is to preserve the semantic meaning of the original messages rather than achieving **verbatim recovery**, which is typically the goal of traditional communication systems. Verbatim recovery is crucial in applications where exact message integrity is required, such as transmitting protocol files, legal documents, financial contracts, or scientific data, where even minor alterations can lead to significant misunderstandings or errors. The system presented in this paper focuses on verbatim message recovery, leveraging the semantic understanding of LMs to achieve this goal.

6 Conclusion and Outlooks

In 1948, Shannon defined the fundamental limit of physical-layer error correction. Decades later, modern communication systems have steadily approached this limit through relentless progress. However, a new question has emerged within the community: is it possible to surpass this upper limit? Recent breakthroughs in LMs suggest an intriguing possibility: advances in these generative models allow them to analyze the linguistic and logical structures inherent in a transmitted message, which serve as an additional source of information beyond the physical signal, for Semantic Error Corrections (SEC).

This work proposes the **Cross-Layer SEC (`CL-SEC`)** framework, an initial exploration into integrating classic physical-layer error correction – where linguistic and semantic considerations are not contributing factors – and application-layer semantic error correction – where bit-level soft information is not contributing – to enhance error correction within a communication system. Unlike previous SEC efforts operated purely at the application layer, `CL-SEC` integrates physical-layer soft information with application-layer semantic context to achieve **verbatim recovery** of the transmitted messages.

For each detected erroneous word, `CL-SEC` identifies candidate word corrections using the word-length metadata, thereby imposing word-level linguistic constraints. The framework then constructs two complementary probability distributions in parallel for candidate word corrections: 1) the physical-layer distribution, derived from bit-wise soft LLRs, incorporating the signal-level evidence; and 2) the application-layer distribution, obtained from LM mask filling, capturing semantic and compositional consistency. These two distributions are next Bayesian-combined in product form to determine a cross-layer posterior probability distribution. The product form in Bayesian combination is justified following both the BP construct and the detailed probability derivations.

While it remains unclear whether linguistic and logical considerations from higher layers can achieve an error-correction performance beyond the Shannon limit, performance benchmarking between `CL-SEC` and its single- and isolated-layer baselines – especially the BCJR algorithm used for lower-layer decoding – highlight an intriguing possibility.

We believe that bridging the gap between physical-layer and semantic-layer error correction could unlock entirely new directions in communication theory, where language structure and meaning play a critical role in ensuring robust transmission. Several key avenues for future research remain open:

- **Source Compression:** The current work does not incorporate source compression, which we believe is likely essential to exploring the potential of surpassing the physical-layer limit with semantic error correction.
- **Theoretical Framework:** This study has largely been experimental and empirical in nature. Developing a rigorous theoretical framework to address this problem not only remains an open challenge but also holds the promise to establish a foundation for a novel field of interdisciplinary research that connects information theory, linguistics, and semantic processing. Our analytical justification for combining information from the physical layer and the application layer – based on BP paradigm and detailed probability derivations – could serve as a potential starting point for this exploration.
- **Word-Length Inference:** In the present study, we assume "known word lengths" to constrain candidate corrections. This requires embedding word-length metadata to the frame header, introducing overhead; moreover, the metadata itself can be corrupted by channel noise. Future work can address these limitations by inferring word boundaries, enabling corrections with accurate word lengths without relying on explicit metadata.
- **Processing Latency:** Introducing post-FEC correction refinement – especially LM inference-based refinement – inevitably adds computational overhead and latency over raw FEC, posing a challenge for latency-sensitive applications. Future work can address this bottleneck to minimize processing latency.
- **Wireless Fading Channels:** Currently, our evaluation is limited to the AWGN channel. While AWGN serves as a fundamental baseline, practical wireless environments are often characterized by fading channels. Future research can extend the framework to fading channels to demonstrate the robustness of error correction.

Acknowledgments

The work was partially supported by the Shen Zhen-Hong Kong-Macao technical program under Grant No. SGDX20230821094359004. The investigation was conducted

in the JC STEM Lab of Advanced Wireless Networks for Mission-Critical Automation and Intelligence funded by The Hong Kong Jockey Club Charities Trust.

References

[1] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv. 1974. Optimal decoding of linear codes for minimizing symbol error rate (Corresp.). *IEEE Trans. Inf. Theory* 20, 2 (Mar. 1974), 284–287.

[2] Shuaishuai Guo, Yanhu Wang, Jia Ye, Anbang Zhang, Peng Zhang, and Kun Xu. 2025. Semantic importance-aware communications with semantic correction using large language models. *IEEE Trans. Mach. Learn. Commun. Netw.* 3 (2025), 232–245.

[3] Jiafu Hao, Chentao Yue, Hao Chang, Branka Vucetic, and Yonghui Li. 2025. Short wins long: Short codes with language model semantic correction outperform long codes. In *Proc. of IEEE GLOBECOM*. 1226–1231.

[4] F.R. Kschischang, B.J. Frey, and H.-A. Loeliger. 2001. Factor graphs and the sum-product algorithm. *IEEE Trans. Inf. Theory* 47, 2 (Feb. 2001), 498–519.

[5] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proc. of ACL*. 7871–7880.

[6] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*. 74–81.

[7] Marc Marone, Orion Weller, William Fleshman, Eugene Yang, Dawn Lawrie, and Benjamin Van Durme. 2025. mmBERT: A modern multilingual encoder with annealed language learning. *arXiv preprint arXiv:2509.06888* (2025).

[8] John G. Proakis and Masoud Salehi. 2001. *Digital communications*. McGraw-Hill New York.

[9] Li Qiao, Mahdi Boloursaz Mashhadi, Zhen Gao, and Deniz Gündüz. 2025. Token-domain multiple access: Exploiting semantic orthogonality for collision mitigation. In *Proc. of IEEE INFOCOM*.

[10] Ron Roth. 2006. *Introduction to coding theory*. Cambridge Univ. Press.

[11] Mohammad Rowshan, Min Qiu, Yixuan Xie, Xinyi Gu, and Jinhong Yuan. 2024. Channel coding toward 6G: Technical overview and outlook. *IEEE Open J. Commun. Soc.* 5 (2024), 2585–2685.

[12] William Ryan and Shu Lin. 2009. *Channel codes: Classical and modern*. Cambridge Univ. Press.

[13] Khalid Sayood. 2017. *Introduction to data compression*. Morgan Kaufmann.

[14] A.J. Viterbi. 1998. An intuitive justification and a simplified implementation of the MAP decoder for convolutional codes. *IEEE J. Sel. Areas Commun.* 16, 2 (1998), 261–264.

[15] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[16] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *Proc. of ICLR*.

A Justification for the Product-Form Bayesian Combination

We aim to the maximize the cross-layer posterior probability $P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big)$ among candidate corrections $\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n$ for each $n \in \mathcal{N}_\mathrm{e}$, given all the information available during the word-level channel decoding and the MLM mask prediction. Analogous to the common assumption that bits 0 and 1 are equally likely, we assume that all the candidates $\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n$ are equally likely in the MLM vocabulary. The cross-layer posterior probability can be written as

$$\begin{aligned} P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big) &= \frac{P\big(\tilde{\mathbf{w}}_{n,s}, \hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\mathrm{MLM}\big)}{P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\mathrm{MLM}\big)} \\ &= \frac{P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big) \cdot P\big(\tilde{\mathbf{w}}_{n,s}|\mathrm{MLM}\big)}{P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\mathrm{MLM}\big)} \\ &\overset{(a)}{\propto} P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big), \end{aligned} \tag{10}$$

where (a) follows from removing the two terms $P(\tilde{\mathbf{w}}_{n,s}|\mathrm{MLM}) = 1/|\mathcal{W}_n|$ and $1/P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\mathrm{MLM}\big)$ that are not functions of $\tilde{\mathbf{w}}_{n,s}$.

The probability $P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big)$ in (10) can be further expressed as

$$\begin{aligned} P\big(\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big) &= P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big) \cdot P\big(\hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big) \\ &\overset{(b)}{\approx} P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}\big) \cdot P\big(\hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big), \end{aligned} \tag{11}$$

where approximation (b) follows from the analysis in Sec. 3.6 that only very little of $\hat{\boldsymbol{\xi}}_n$ is related to $\hat{\mathbf{c}}_n$. Specifically, $\hat{\boldsymbol{\xi}}_n$ is the contextual information provided by the surrounding words about the masked word, and the decoding of the surrounding words are based largely on bits not within $\hat{\mathbf{c}}_n$ (only the few boundary bits in $\hat{\mathbf{c}}_n$ contain minute information about the adjacent words). Thus, we assume $\hat{\mathbf{c}}_n$ in $P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big)$ largely depends on $\tilde{\mathbf{w}}_{n,s}$ only. That is, $P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big) \approx P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}\big)$ — the bits in $\tilde{\mathbf{w}}_{n,s}$ and the physical channel noise largely determines $\hat{\mathbf{c}}_n$ which is derived from the received physical signal. **This is where our key approximation occurs.**

The first probability in (11) satisfies $P\big(\hat{\mathbf{c}}_n|\tilde{\mathbf{w}}_{n,s}\big) \propto P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n\big)$ given the equally likely candidates $\tilde{\mathbf{w}}_{n,s} \in \mathcal{W}_n$. The second probability in (11) can be written as

$$\begin{aligned} P\big(\hat{\boldsymbol{\xi}}_n|\tilde{\mathbf{w}}_{n,s}, \mathrm{MLM}\big) &= P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big) \cdot \frac{P\big(\hat{\boldsymbol{\xi}}_n|\mathrm{MLM}\big)}{P\big(\tilde{\mathbf{w}}_{n,s}|\mathrm{MLM}\big)} \\ &\overset{(c)}{\propto} P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big), \end{aligned} \tag{12}$$

where (c) follows from removing the two terms that are not functions of $\tilde{\mathbf{w}}_{n,s}$. With the above results, for each $n \in \mathcal{N}_\mathrm{e}$, the cross-layer posterior probability can be expressed as

$$P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n, \hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big) \propto P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\mathbf{c}}_n\big) \cdot P\big(\tilde{\mathbf{w}}_{n,s}|\hat{\boldsymbol{\xi}}_n, \mathrm{MLM}\big),\ s \in \mathcal{S}_n, \tag{13}$$

which coincides with (8) following BP operated in the factor graph (Fig. 3). Therefore, we have justified the product form in Bayesian combination as an approximation and we have also explicitly pointed out where the approximation occurs.

B Canonical Huffman Coding for Word-Length Information Compression

This paper studies five correction schemes and Schemes 2–5 (see Sec. 4.1) requires word-length metadata. To reduce overhead of the word-length metadata, we employ Huffman coding [13]. Huffman coding is a lossless compression algorithm that assigns shorter binary bits to more frequent symbols and longer bits to less frequent ones, thereby minimizing the average codeword length. In the basic Huffman coding framework, the entire Huffman tree must be transferred to the decoder, resulting in significant codebook overhead. Canonical Huffman coding is a standardized variant that reorganizes codewords such that only their lengths need to be transmitted. This canonical form enables deterministic reconstruction of the codebook at the decoder while reducing codebook overhead.

In this work, we employ canonical Huffman coding to efficiently compress the word-length information of the message. The compressed word lengths are embedded in the frame header as metadata. For each word length L_n, $n \in \mathcal{N}$ in the message, we denote the length of its associated canonical Huffman codeword by Θ_n (in bits). Thus, we require $\sum_{n\in\mathcal{N}} \Theta_n$ bits to transmit word lengths. In addition, for codebook delivery, transmitting the length of each canonical Huffman codeword suffices, as mentioned earlier. We use

Table 2: Word-length statistics and the resulting canonical Huffman codebook.

Word length (in letters)	3	4	2	5	6	8	7	10	1	9	12	11
Word count	31	21	19	14	10	8	7	4	4	2	1	0
Canonical Huffman codeword	00	01	100	101	1100	1101	1110	11110	111110	1111110	1111111	None
Codeword length (in bits)	2	2	3	3	4	4	4	5	6	7	7	"0"

fixed-length binary coding for codebook delivery and assume such delivery requires additional Φ bits. The codeword lengths associated with each word length can be transmitted in a preset word-length order. The decoder can thus rely such order to determine each pair of codeword length and word length, based on which the decoder can further construct the complete canonical Huffman codebook uniquely. Relative to the payload $\sum_{n\in\mathcal{N}} K_n$ bits for word transmission, the header overhead ρ is defined as

$$\rho = \frac{\sum_{n\in\mathcal{N}} \Theta_n + \Phi}{\sum_{n\in\mathcal{N}} K_n}. \tag{14}$$

To illustrate coding details and the compression efficiency, consider the following compression example. One English message is:

```
Modern climbers try to climb mountains by a route which will give
them good sport, and the more difficult it is, the more highly it is
regarded. In the pioneering days, however, this was not the case at
all. The early climbers were looking for the easiest way to the top,
because the summit was the prize they sought, especially if it had
never been attained before. It is true that during their adventures
they often faced difficulties and dangers of the most perilous
nature, equipped in a manner which would make modern climbers shudder
at the thought, but they did not go out of their way to court such
excitement. They had a single aim, a solitary goal – the top!
```

After removing punctuation and spaces, the message contains $N = 121$ words with 532 letters in total, corresponding to $\sum_{n\in\mathcal{N}} K_n = 4256$ bits in the payload. The word-length statistics and the resulting canonical Huffman codebook are listed in Table 2. With this codebook, transmitting $N = 121$ word lengths in this message requires $\sum_{n\in\mathcal{N}} \Theta_n = 368$ bits. Regarding codebook delivery, we represent the length of each Huffman codeword using a fixed-length binary code. Note that there is no word in this message with word length 11. We reserve codeword length 0 for such nonexistent word length. Consequently, we encode the codeword lengths varying from the minimum 0 to the maximum 7 using 3 bits, covering lengths $0, 1, \ldots, 7$. Thus, transmitting 11 codeword lengths and a reserved codeword length 0 requires an additional $\Phi = 36$ bits, which whereas is much smaller than $\sum_{n\in\mathcal{N}} \Theta_n = 368$ bits for word-length transmission. The codeword lengths corresponding to different word lengths can be transmitted in a preset order, e.g., in the ascending word-length order. With such order, the decoder can determine the codeword length for each word length, and can further construct the canonical Huffman codebook uniquely. Summing two parts of the header overhead, the total overhead for word-length transmission becomes $\sum_{n\in\mathcal{N}} \Theta_n + \Phi = 404$ bits. Relative to the payload $\sum_{n\in\mathcal{N}} K_n = 4256$ bits, the header overhead percentage ρ is **9.49%** using (14). This overhead is small in practical communication systems.

The compression efficiency is further extensively evaluated over a message corpus (including the earlier message example). To cater for diverse real-word cases, we include 550 passages in this corpus, consisting of the 50 passages from English textbooks (see Sec. 4.1) and the 500 passages in Brown Corpus. The statistics of the header overhead are listed in Table 3. Across the corpus, **98.73%** of the passages incur an overhead of **less than 10%**, indicating that the header overhead is small in most practical cases.

Table 3: Statistics of the header overhead across the corpus.

	Minimum	Maximum	Mean	Std dev
Overhead ρ	7.69%	10.77%	8.69%	0.43%